

AI Pioneers

CCA-F | Module 5

Lab 5.1

Managing Context: Preservation, Optimization & Escalation

Scenario: An E-commerce Customer Support Agent (multi-turn, multi-order session)

Module	M5 — Context Management & Reliability
Lab type	Hands-on · Self-paced or Instructor-Led · Claude API (Python)
Duration	~45 minutes (hands-on)
Environment	Blue Labs VM · Python 3.10+ · <code>anthropic</code> + <code>python-dotenv</code> · <code>ANTHROPIC_API_KEY</code>
Sections	S1 (Context Preservation & Optimization) S2 (Escalation & Ambiguity Resolution Patterns)

Lab Objectives

By the end of this lab you will be able to:

- **Preserve persistent case facts across long sessions** — pin identity, tier, and active order id into a `[CASE FACTS]` block that is re-injected into the system prompt every turn, so the agent never loses who it is talking to even after the early messages are trimmed.
- **Optimize tool outputs before they reach the model** — route every tool result through `optimize(tool_name, raw)` so a hundred-row order dump becomes a handful of relevant fields. Saves tokens, keeps the answer un-buried, and stops noise from pushing earlier turns out of the context window.
- **Escalate ambiguity instead of guessing** — when a request is under-specified ("cancel my order" with two open orders), the system prompt instructs the agent to ASK rather than pick one at random, turning a real, costly mistake into a one-line clarifying question.

1. Overview

A short demo agent runs fine on three turns. A real support session runs eighteen, and the failure modes look different: it forgets the customer id, it pastes hundred-row order dumps into the model, and it guesses on ambiguous commands

instead of asking. This lab applies three production techniques to a single e-commerce support agent so the same code works at turn three and turn thirty. First you pin identity into a [CASE FACTS] block that gets re-injected into the system prompt every turn — the model never has to dig through history to remember who it is talking to. Next you route every tool result through an `optimize()` function that keeps only the fields the current intent needs, so a bulky `lookup_orders` response shrinks to a few key columns. Finally you set the system prompt to ASK on ambiguous requests rather than guess, so "cancel my order" against two open orders triggers a clarifying question instead of a costly wrong-order cancellation. Everything runs as one `main.py` with three labelled demos.

SN	Demo	What you will do	Core idea
Ex 1	S1 — Persistent case facts	Pin identity in a <code>CaseFacts</code> store and append it as a [CASE FACTS] block to the system prompt every turn.	Critical facts survive even when the message history is trimmed.
Ex 2	S1 — Tool output optimization	Wrap raw tool results through <code>optimize(tool, raw)</code> so only the relevant fields reach the model.	Tokens stay cheap; the signal stays un-buried.
Ex 3	S2 — Ambiguity escalation	Set the system prompt to ask a clarifying question when the user's request maps to more than one record (e.g. "cancel my order" with two open orders).	Asking once is cheaper than picking wrong.

2. Scenario

You are building the customer-support agent for an online retail company. The agent answers multi-turn questions about orders, returns, refunds, and shipping. A real session runs ten to twenty turns, the customer asks about several topics, and the agent calls a couple of internal tools along the way (`lookup_orders`, `get_order_details`). The same agent has to behave well at all three pinch points — long sessions, bulky tool outputs, ambiguous commands — without bolting on a different fix for each.

Your running example is a Gold-tier customer, Aarti Sharma (C-1001), who has three orders on file (one delivered, one shipped, one processing). The three demos walk her through the failure modes a naïve agent would hit and show what the production fix looks like in code.

You will:

- Build a `CaseFacts` store and have the chat loop re-inject `facts.as_system_block()` into the system prompt on every API call, so customer id and tier survive 18 turns of small talk.
- Implement `optimize(tool_name, raw)` against a per-tool whitelist of `RELEVANT_FIELDS` and route every `tool_result` through it before sending it back to the model.
- Phrase the system prompt so an ambiguous request ("cancel my order") triggers a clarifying question rather than a guess — verify against a customer who has two open orders.

The three techniques at a glance

```
preserve    -> [CASE FACTS] block re-injected into system prompt every turn
optimize    -> optimize(tool, raw) keeps only whitelisted fields per tool
```

```
escalate    -> system prompt: "if ambiguous, ASK a clarifying question"
```

Why these three together?

Each one fixes a different real failure mode and they reinforce each other. Case facts keep the agent from re-asking what it already knows, so the conversation stays short. Tool optimization keeps each response small, so the case facts don't get crowded out of the context window. Ambiguity escalation keeps the agent honest in exactly the situations where the first two fixes saved you tokens but didn't save you from a wrong guess. Drop any one of them and a long session degrades in the others: a verbose tool output evicts the case facts; missing facts force the model to guess; a guess on an ambiguous order is the kind of mistake you pay for at the customer-service desk.

3. Pre-requisites

3.1 Skilljar Videos — Completed in Earlier Modules

Course	Lessons to watch	Covers
Introduction to Agent Skills	Configuration and multi-file skills	S1
Introduction to Subagents	Using subagents effectively · Designing effective subagents	S2
Introduction to Agent Skills	Skills vs. other Claude Code features	S2

Module 5 introduces no new Skilljar courses — it builds on lessons you completed in Modules 1 and 2. Revisit the specific lessons above before the session; the instructor will not re-teach them. Sections S5 and S6 (not covered in this lab) have no dedicated Skilljar lesson at all; they are built in their respective instructor-led labs.

3.2 Environment Check

Run these checks before the session starts. If any step fails, contact the instructor or check the Blue Labs SOP.

- Start your Blue Labs VM: open <https://www.bluelabs.studio/>, locate your VM, click Start, then Connect.
- Unzip the lab bundle and enter it:

```
unzip lab_5_1_context_management.zip
cd lab_5_1_context_management
```

- Create a Python environment and install both dependencies:

```
python -m venv .venv && source .venv/bin/activate    # Windows:
.venv\Scripts\activate
pip install -r requirements.txt                      # anthropic + python-dotenv
```

- Add your API key (the lab uses `python-dotenv` — paste your key into `.env`):

```
cp .env.example .env          # paste your key into .env
# .env content:
# ANTHROPIC_API_KEY=sk-ant-your-key-here
# MODEL_NAME=claude-sonnet-4-6
```

- Confirm your setup: run `python main.py`. Until you complete the three TODOs it stops at the first one (Demo 1's `as_system_block`) with a clear message — that still confirms the modules import and your dependencies loaded. With the TODOs done it runs all three demos in sequence:

```
python main.py
```

Model, cost & safety

The script reads the model from `MODEL_NAME` (default `claude-sonnet-4-6`) and the key from `ANTHROPIC_API_KEY`, both via `python-dotenv`. A full run is about a dozen short turns plus two tool calls — costs are minimal. The mock data in `sample_data.py` contains one fictional customer (Aarti Sharma, C-1001) and three sample orders, all dummy values — no PII, no real account data.

4. Demos

Open `main.py`. Each demo has one TODO to complete (see the module docstring); the demos are clearly labelled with banner prints, and once your TODOs are done, running `python main.py` executes them in sequence. The supporting modules (`case_facts.py`, `tool_optimizer.py`, `sample_data.py`) are short — every function has a docstring explaining why the technique matters, not just what the code does.

Demo 1: Preserve persistent case facts (Preservation) ~15 min

Background

Long sessions evict early messages. If the customer's id or tier only appeared in turn 2, by turn 18 it is either out of the window or its attention is weak. The production fix is a small, always-visible `[CASE FACTS]` block re-injected into the system prompt every turn. The model never has to dig through history to remember who it is talking to.

Task

Open `case_facts.py` and implement the `as_system_block()` TODO in the `CaseFacts` class. Then trace through `demo_1_case_facts()` in `main.py`: three unrelated turns about store hours, shipping, and payment, followed by "what is my customer ID and what tier am I?" — the agent must still know.

Step 1 — The CaseFacts store

A tiny key-value store with one trick: a `as_system_block()` method that renders the facts into a single chunk of text. Keep it short — every token here is paid for on every API call.

```
class CaseFacts:
    """A tiny key-value store of facts that must survive every turn."""
    def __init__(self):
        self._facts: dict[str, str] = {}

    def set(self, key: str, value: str) -> None:
        self._facts[key] = value

    def as_system_block(self) -> str:
        if not self._facts:
            return ""
        lines = ["[CASE FACTS - these are confirmed and must be preserved]"]
        for k, v in self._facts.items():
            lines.append(f"- {k}: {v}")
        return "\n".join(lines)
```

Step 2 — Re-inject every turn

Inside the chat loop, append the facts block to `system_prompt` before `messages.create`. The facts are pinned, not floating somewhere in the message history:

```
def chat(messages: list, facts: CaseFacts) -> str:
    system_prompt = SYSTEM_BASE
    facts_block = facts.as_system_block()
    if facts_block:
        system_prompt = system_prompt + "\n\n" + facts_block
    # ... messages.create(system=system_prompt, ...)
```

Step 3 — Tell the model the block is authoritative

The base system prompt explicitly grants the [CASE FACTS] block authority so the model uses it instead of asking again:

```
SYSTEM_BASE = (
    "You are a helpful support agent for an online retail company. "
    "Be concise. If a request is ambiguous (e.g. the customer has multiple "
    "open orders and didn't specify which), ASK a clarifying question "
    "instead of guessing. Always rely on the [CASE FACTS] block - those "
    "values are authoritative and you do not need to ask for them again."
)
```

Step 4 — Run and confirm

```
python main.py
```

Expected: at turn 4 the agent answers "your customer ID is C-1001 and you are on the Gold tier" without having to re-ask, even though no earlier turn ever mentioned those values. The [CASE FACTS] block did the heavy lifting silently.

What good looks like

The agent's reply at the "quick check" turn names the customer id and tier verbatim — C-1001 and Gold — without a tool call, because they were in the system prompt all along. Try commenting out the `facts_block` line and re-run: the agent will either ask the customer to identify themselves or guess, depending on the turn count.

Reflection Questions

- Why put case facts in the **system prompt** rather than as an early user/assistant turn or as a tool response?
- Every token in `as_system_block()` is paid for on every API call. Which kinds of facts belong here, and which kinds belong in normal turns or tools?
- How would you decide which facts to evict from the block over a session that has been running for an hour?

Demo 2: Optimize tool outputs (Optimization) ~15 min

Background

A raw `lookup_orders` call against a real database returns full order records: line items, SKUs, addresses, dozens of columns each, hundreds of rows for an active customer. Pasting all of that into the conversation burns tokens, buries the few useful fields under noise, and pushes earlier turns (including the [CASE FACTS] block) closer to eviction. The fix is a whitelist per tool: `optimize(tool, raw)` keeps only the fields the current intent needs.

Task

Open `tool_optimizer.py` and read the `RELEVANT_FIELDS` table plus the `optimize()` function — this is the TODO you complete. Then trace `demo_2_tool_optimization()` — it prints the RAW output, then the OPTIMIZED output, then lets the agent answer the user's request using only the optimized version.

Step 1 — The whitelist

Per-tool list of the fields you actually want the model to see. Anything not listed is dropped:

```
# Fields we actually care about for each tool. Anything not in this list
# is dropped before the result reaches the model.
RELEVANT_FIELDS = {
    "lookup_orders": ["order_id", "status", "placed_on", "total"],
    "get_order_details": ["order_id", "status", "placed_on", "total", "items"],
}
```

Step 2 — Implement optimize()

Trim a tool's raw output to just the whitelisted fields, in the same shape (list or dict) it came in. Unknown tools pass through untouched (in production you'd log a warning so unconfigured tools don't sneak past):

```
def optimize(tool_name: str, raw_result):
    keep = RELEVANT_FIELDS.get(tool_name)
    if keep is None:
        return raw_result # no rule — pass through (consider logging)
    if isinstance(raw_result, list):
        return [{k: row[k] for k in keep if k in row} for row in raw_result]
    if isinstance(raw_result, dict):
        return {k: raw_result[k] for k in keep if k in raw_result}
    return raw_result
```

Step 3 — Wire it into the chat loop

Every tool call goes through `run_tool()`, which calls the raw mock-DB function and then `optimizes` the result before returning JSON to the model:

```
def run_tool(name: str, args: dict) -> str:
    """Execute a tool, optimize the result, return JSON for the model."""
    if name == "lookup_orders":
        raw = get_orders_for_customer(args["customer_id"])
    elif name == "get_order_details":
        from sample_data import ORDERS
        raw = ORDERS.get(args["order_id"]) or {}
    else:
        raw = {"error": f"unknown tool {name}"}
    trimmed = optimize(name, raw)
    return json.dumps(trimmed)
```

Step 4 — See the before-and-after

```
python main.py # Demo 2 prints RAW vs OPTIMIZED side-by-side
```

Expected: the RAW block shows the full mock order records (`customer_id`, `items` with SKUs and quantities — all the noise). The OPTIMIZED block shows only `order_id`, `status`, `placed_on`, `total` — exactly what the model needs to answer "list my orders, just status and total".

What good looks like

The OPTIMIZED block is visibly smaller (typically 3–5× fewer characters than RAW for this tool), the agent's reply uses only those fields, and you can see in the trace that the model never had to read SKUs or item names to answer a question about status and total. Adding a new tool means one line in `RELEVANT_FIELDS` — not a new code path.

Reflection Questions

- Why is the whitelist keyed on tool name rather than on the model's intent? When would you want it keyed on intent instead?
- What goes wrong if you whitelist too few fields? Too many?
- The optimizer is content-blind — it trims by field name. When would you reach for content-aware summarization (e.g. summarizing a long text field) on top of this?

Demo 3: Escalate ambiguity instead of guessing (Escalation) ~15 min

Background

A customer says "cancel my order" and has two open orders. Cancelling the wrong one is a real, costly mistake — refunds, callbacks, escalations. The right behaviour is to ASK which one. This isn't a code-path fix in your agent; it is a single sentence in the system prompt that the model honours throughout the session.

Task

Complete the `SYSTEM_BASE` constant (add the ASK-on-ambiguity rule) in `main.py`, then trace `demo_3_escalate_ambiguity()`: a customer with two open orders (O-9002 Shipped, O-9003 Processing) sends "please cancel my order" — and the agent must respond with a clarifying question, not a tool call to cancel.

Step 1 — The instruction that does the work

The escalation behaviour lives in the system prompt — explicit, short, and tied to a concrete trigger (multiple open orders):

```
SYSTEM_BASE = (
    "You are a helpful support agent for an online retail company. "
    "Be concise. If a request is ambiguous (e.g. the customer has multiple "
    "open orders and didn't specify which), ASK a clarifying question "
    "instead of guessing. Always rely on the [CASE FACTS] block - those "
    "values are authoritative and you do not need to ask for them again."
)
```

Step 2 — Set up an ambiguous scenario

Pin `customer_id` into the case facts so the agent knows whose orders to consider, but do **not** name a specific order. Send the bare request:

```
facts = CaseFacts()
facts.set("customer_id", "C-1001")

open_orders = get_open_orders_for_customer("C-1001")
print(f"Customer has {len(open_orders)} open orders: "
      f"{[{o['order_id'] for o in open_orders}]}")

messages = [{"role": "user", "content": "Please cancel my order."}]
```



```
reply = chat(messages, facts)
print(f"AGENT: {reply}")
```

Step 3 — Run; verify the agent asks, doesn't guess

```
python main.py      # Demo 3 — the agent must ask which order
```

Expected: the agent calls `lookup_orders` to discover the two open orders, then replies with a short clarifying question naming both ids — "You have two open orders, O-9002 (Shipped) and O-9003 (Processing). Which one would you like to cancel?" — not a unilateral cancellation.

What good looks like

The agent's reply is a question — it names the candidate orders by id and status, and does not invoke any cancel-style tool. Remove the "if a request is ambiguous ... ASK" sentence from `SYSTEM_BASE` and re-run: the agent will typically pick one order or ask in a vague way, which is exactly the failure mode this technique exists to prevent.

Reflection Questions

- Why is the escalation rule in the **system prompt** rather than enforced in code (e.g. a guard around the cancel tool)?
- What other ambiguous-request shapes should this trigger on (refunds, address changes, returns)? How would you list them without making the system prompt sprawl?
- If the customer ignores the clarifying question and just says "the second one", how should the agent disambiguate that — and what role do the case facts play?

5. Debrief & Reflection

5.1 Self-Check Before You Leave

Answer each question without looking back.

#	Question
1	Why re-inject the <code>[CASE_FACTS]</code> block into the system prompt every turn instead of saving the early messages once and trusting the model to remember?
2	Which fields belong in <code>CaseFacts</code> , and which belong in ordinary chat history or tool calls?
3	How does <code>optimize(tool, raw)</code> differ from a content-aware summarizer, and when would you reach for each?

#	Question
4	Why is the ambiguity-escalation rule expressed in the system prompt rather than enforced as a code guard around the cancel tool?
5	How do the three techniques reinforce each other? Pick any pair and explain the dependency.

5.2 Common Mistakes

Mistake	Why it matters and what to do instead
Stuffing case facts into the first user turn.	As soon as the history gets trimmed the facts go with it. Put them in the system prompt and rebuild that block on every API call.
Letting raw tool output reach the model.	Hundreds of rows × dozens of fields evicts earlier turns. Route every tool through <code>optimize(tool, raw)</code> with a per-tool whitelist.
Over-fitting the whitelist to one demo.	A whitelist that only keeps <code>status</code> breaks the next intent that needs <code>total</code> . Whitelist the union of fields any reasonable intent on that tool needs, not the bare minimum.
Encoding escalation as a code guard.	Guarding <code>cancel_order()</code> in Python catches one tool; the model still picks the wrong one elsewhere. Put the ASK on ambiguity rule in the system prompt so it applies to every intent.
Bloating the [CASE FACTS] block.	Every token in <code>as_system_block()</code> is paid on every call. Keep it to identity-class facts (customer id, tier, active order). Long lists belong in tool output, not the system prompt.
Telling the model the block exists but not that it's authoritative.	Without an explicit "these values are authoritative and you do not need to ask for them again" sentence, the model often asks anyway out of politeness. State it.
Silently swallowing unknown tools in <code>optimize()</code> .	A new tool added without a whitelist entry slips through with its full payload — exactly the failure mode this is meant to prevent. Log a warning in the <code>keep is None</code> branch.

6. Quick Reference

6.1 Pin facts; re-inject every turn

```
class CaseFacts:
    def __init__(self): self._facts: dict[str, str] = {}
    def set(self, k, v): self._facts[k] = v
    def as_system_block(self) -> str:
        if not self._facts: return ""
        lines = ["[CASE FACTS - these are confirmed and must be preserved]"]
        for k, v in self._facts.items():
            lines.append(f"- {k}: {v}")
        return "\n".join(lines)
```

```
# In the chat loop, on EVERY messages.create call:
system_prompt = SYSTEM_BASE + "\n\n" + facts.as_system_block()
```

6.2 Trim tool output to a per-tool whitelist

```
RELEVANT_FIELDS = {
    "lookup_orders": ["order_id", "status", "placed_on", "total"],
    "get_order_details": ["order_id", "status", "placed_on", "total", "items"],
}

def optimize(tool_name, raw):
    keep = RELEVANT_FIELDS.get(tool_name)
    if keep is None:
        # log a warning here in production
        return raw
    if isinstance(raw, list):
        return [{k: row[k] for k in keep if k in row} for row in raw]
    if isinstance(raw, dict):
        return {k: raw[k] for k in keep if k in raw}
    return raw
```

6.3 The system-prompt rules that do the work

```
SYSTEM_BASE = (
    "You are a helpful support agent for an online retail company. "
    "Be concise. If a request is ambiguous (e.g. the customer has multiple "
    "open orders and didn't specify which), ASK a clarifying question "
    "instead of guessing. Always rely on the [CASE FACTS] block - those "
    "values are authoritative and you do not need to ask for them again."
)
```

6.4 The three techniques as one decision matrix

```
# Which failure are you fixing?
#
#   forgets identity mid-session   -> [CASE FACTS] in system prompt (Demo 1)
#   tool output crowds out turns  -> optimize(tool, raw) whitelist (Demo 2)
#   guesses on ambiguous command  -> ASK-on-ambiguity in system prompt (Demo 3)
#
# All three apply on every turn. Removing any one re-opens that failure mode.
```

6.5 File Inventory

File	Role	Purpose
<code>main.py</code>	entry	Chat loop, tool dispatch, three labelled demos (<code>demo_1_case_facts</code> , <code>demo_2_tool_optimization</code> , <code>demo_3_escalate_ambiguity</code>).
<code>case_facts.py</code>	D 1	The <code>CaseFacts</code> store and its <code>as_system_block()</code> renderer.
<code>tool_optimizer.py</code>	D 2	The <code>RELEVANT_FIELDS</code> whitelist and the <code>optimize()</code> function.
<code>sample_data.py</code>	data	Mock <code>CUSTOMERS</code> / <code>ORDERS</code> dicts (Aarti Sharma, C-1001, three orders) plus the lookup helpers.
<code>.env.example</code> , <code>requirements.txt</code>	setup	API key template, model override, and the two pinned dependencies (<code>anthropic</code> , <code>python-dotenv</code>).

6.6 Further Reading

- Anthropic API overview: <https://docs.claude.com/en/api/overview>
- Tool use guide: <https://docs.claude.com/en/docs/build-with-claude/tool-use>
- Skilljar (Module 1) — **Introduction to Agent Skills**: Configuration and multi-file skills (S1 supporting)
- Skilljar (Module 1) — **Introduction to Subagents**: Using subagents effectively; Designing effective subagents (S2)
- Skilljar (Module 1) — **Introduction to Agent Skills**: Skills vs. other Claude Code features (S2 supporting)